
Übungsblatt 4

Abgabe: 18.05.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Aufgabe 13 (*Schnittstellen-Objekte und Arrays, 15 Minuten*)

a) (**, 7 Minuten)

- Schreiben Sie zuerst eine Methode, die ein `CharSequence`-Objekt als Parameter erwartet. Gehen Sie davon aus, dass dieser Parameter die Form `"dateiname.dateityp"` hat. Die Methode soll die Dateinamenserweiterung `"dateityp"` als `CharSequence`-Objekt zurückgeben.
- Dazu soll die Methode nach dem letzten Vorkommen eines Punkts `"."` suchen, und ein `CharSequence`-Objekt mit allen Zeichen ab diesem Punkt (inklusive des Punkts) zurückgeben.
- Enthält das `CharSequence`-Objekt keinen Punkt, so soll die leere Zeichenkette zurückgegeben werden.
- Erstellen Sie in der `main()`-Methode jeweils ein Objekt von Typ `String`, `StringBuffer` und `StringBuilder` mit einem Dateinamen als Inhalt.
- Geben Sie für jedes der drei Objekte die Dateinamenserweiterung aus.
- Einen passenden Methodenkopf entwerfen Sie selbst.

b) (**, 8 Minuten)

Schreiben Sie ein Programm, das

- ein `CharSequence`-Array anlegt, das groß genug ist, um alle Kommandozeilenparameter aufzunehmen,
- mit der Klassenmethode `arraycopy` der `System`-Klasse die Kommandozeilenparameter in das angelegte `CharSequence`-Array kopiert,
- ein `String`-Objekt mit dem Wert `"matik"` anlegt, das von einer `CharSequence`-Variable `str` referenziert wird und
- für jeden Eintrag des `CharSequence`-Arrays überprüft, ob diese eine Teilstring enthalten, die `str` entspricht. Im positiven Fall soll die jeweils erste Position des Vorkommens ausgegeben werden.

Verwenden Sie ausschließlich Methoden der Klasse `CharSequence`, um die notwendigen Vergleiche durchzuführen!

Aufgabe 14 ** (Mehrdimensionale Arrays, 41 Minuten)

In dieser Aufgabe sollen Sie eine Programmklasse zur Verwaltung von Matrizen mit ganzzahligen Einträgen erstellen, die neben der `main`-Methode fünf weitere Methoden

- zwei mal `newMatrix`,
- `isMatrix`,
- `printMatrix` und
- `addMatrix`

nach den Vorgaben der unteren Teilaufgaben enthält. Eine Matrix soll dabei durch ein zweidimensionales `int`-Array repräsentiert werden.

In allen Aufgaben ist mit Ausgabe die Ausgabe auf der Kommandozeile (= Standardausgabe) gemeint.

a) (**, die Methoden `newMatrix`, 7 Minuten)

Implementieren Sie zwei Klassenmethoden `newMatrix`, die sich durch die übergebenen Parameter unterscheiden. Eine Methode soll dabei dafür verwendet werden, eine Matrix mit n Zeilen und m Spalten mit zufälligen ganzzahligen Werten zwischen 0 und 100 (einschließlich) zu erstellen und zurückzugeben. Die andere Methode soll für die Erstellung von quadratischen Matrizen mit n Zeilen und Spalten mit zufälligen ganzzahligen Werten zwischen 0 und 100 (einschließlich) verwendet werden. Die Köpfe der Methoden legen Sie geeignet selbst fest.

b) (**, die Methode `isMatrix`, 5 Minuten)

Implementieren Sie eine Klassenmethode `isMatrix`, die überprüft, ob ein übergebenes zweidimensionales `int`-Array eine Matrix ist, d.h. alle Zeilen gleich lang sind. Den Kopf der Methode legen Sie geeignet selbst fest.

c) (**, die Methode `printMatrix`, 6 Minuten)

Implementieren Sie eine Klassenmethode `printMatrix`, die zuerst überprüft, ob die übergebene Matrix wohlgeformt ist, und im positiven Fall die Matrix übersichtlich formatiert ausgibt. Ist die Matrix nicht wohlgeformt, soll eine passende Fehlermeldung auf der Standardfehlerausgabe ausgegeben werden. Den Kopf der Methode legen Sie geeignet selbst fest.

d) (**, die Methode `addMatrix`, 8 Minuten)

Implementieren Sie eine Klassenmethode `addMatrix`, die zwei Matrizen addieren soll. Wenn eine der Matrizen nicht wohlgeformt ist oder die Dimensionen der Matrizen nicht übereinstimmen, soll eine entsprechende Fehlermeldung auf der Standardfehlerausgabe ausgegeben und `null` zurückgegeben werden. Ansonsten soll eine neue Matrix erstellt werden, deren Zellen durch Addition der entsprechenden Zellen der übergebenen Matrizen entsteht:

$$(c_{i,j})_{1 \leq i \leq n, 1 \leq j \leq m} := (a_{i,j})_{1 \leq i \leq n, 1 \leq j \leq m} + (b_{i,j})_{1 \leq i \leq n, 1 \leq j \leq m}$$

Den Kopf der Methode legen Sie geeignet selbst fest.

e) (**, die Methode *main*, 15 Minuten)

In der *main*-Methode

- erzeugen Sie zuerst zwei Matrizen *a* und *b* mit 3 Zeilen und 4 Spalten sowie eine Matrix *c* mit 3 Zeilen und 3 Spalten mit zufälligen ganzzahligen Einträgen,
- erzeugen dann ein zweidimensionales **int**-Array *d* mit zwei Zeilen, die erste mit 3 Spalten und die zweite mit 4 Spalten und befüllen dieses manuell mit zufälligen ganzzahligen Einträgen zwischen 0 und 100 (einschließlich),
- versuchen, alle vier mit **printMatrix** auszugeben,
- versuchen, die Summe von *a* und *b* zu berechnen und auszugeben,
- versuchen, die Summe von *a* und *c* zu berechnen und auszugeben und
- versuchen, die Summe von *a* und *d* zu berechnen und auszugeben.

Hinweis: Wenn Sie Ihr Programm in Ihrer IDE ausführen, kann es passieren, dass die Reihenfolge der Ausgaben auf **System.out** und **System.err** durcheinander geraten. Wenn Sie Ihr Programm direkt in der Konsole mit `javac Matrix.main.java` und `java Matrix.main` ausführen, sollte die Reihenfolge der Ausgaben korrekt sein.

Aufgabe 15 ** (Klassen für Zeichenketten benutzen, 20 Minuten)

In den folgenden Teilaufgaben sollen Sie jeweils eine Programmklasse implementieren.

In allen Aufgaben ist mit Ausgabe die Ausgabe auf der Kommandozeile (= Standardausgabe) gemeint.

a) (**, Klasse *StringBuilder*, 5 Minuten)

Schreiben Sie ein übersetzbares Java-Programm, das nur unter Verwendung von Methoden der Klasse *StringBuilder*

- ein *StringBuilder*-Objekt *sb* mit dem Inhalt "Hallo Welt!" erzeugt,
- das Wort "Welt" durch "Java" ersetzt,
- am Anfang des Strings ">>> " einfügt und
- den resultierenden String auf der Kommandozeile ausgibt.

Die Ausgabe soll also wie folgt aussehen:

```
>>> Hallo Java?
```

b) (**, Klasse *StringBuilder*, 6 Minuten)

Schreiben Sie ein übersetzbares Java-Programm, das nur unter Verwendung von Methoden der Klasse *StringBuilder*

- ein *StringBuilder*-Objekt *sb* mit leerem Inhalt erzeugt,
- die Kommandozeilenparameter in **umgekehrter Reihenfolge** (also vom letzten zum ersten) durchläuft und jeden Parameter gefolgt von einem Leerzeichen an *sb* anhängt,
- das letzte (überschüssige) Leerzeichen aus *sb* entfernt,
- die Länge des resultierenden Strings in einer eigenen Zeile ausgibt,
- den String selbst in einer eigenen Zeile ausgibt.

Für die Kommandozeilenparameter "Eins", "Zwei", "Drei" soll die Ausgabe also wie folgt aussehen:

```
14
```

```
Drei Zwei Eins
```

c) (****, Klasse **String**, 7 Minuten)

Schreiben Sie ein übersetzbare Java-Programm, das nur unter Verwendung von **String**-Methoden

- alle Kommandozeilenparameter mit genau einem Methodenaufruf zu einem einzigen String **input** zusammenfügt, wobei die einzelnen Parameter jeweils durch genau ein Leerzeichen getrennt sein sollen,
- überprüft, ob **input** mit einem Großbuchstaben beginnt (Tipp: verwenden Sie eine geeignete Methode der Klasse **Character**) und eine passende Fehlermeldung auf der Standard-*Fehlerausgabe* ausgibt und das Programm beendet, wenn dies nicht der Fall ist,
- überprüft, ob **input** mit einem Punkt '.', Ausrufezeichen '!' oder Fragezeichen '?' endet und eine passende Fehlermeldung auf der Standard-*Fehlerausgabe* ausgibt und das Programm beendet, wenn dies nicht der Fall ist,
- die Anzahl der Wörter in **input** mit einer geeigneten Methode der Klasse **String** ermittelt und ausgibt und
- den String komplett in Großbuchstaben konvertiert und ausgibt.

Für die Kommandozeilenparameter "Das", "ist", "ein", "Test." soll die Ausgabe also wie folgt aussehen:

```
4
DAS IST EIN TEST.
```

Aufgabe 16 (*Wissensfragen (API, Klassen, Schnittstellen)*, 18 Minuten)

Beantworten Sie möglichst knapp und genau in einem Satz die folgenden Fragen:

a) (***, Die Klasse **String**, 8 Minuten)

1. Wie lautet der vollständige Kopf der Klasse?
2. Beschreiben Sie in einem Satz die Objekte der Klasse.
3. Ist die Klasse final?
4. Ist die Klasse abstrakt?
5. Welche Schnittstellen implementiert die Klasse (direkt und indirekt)?
6. Welche Oberklasse hat die Klasse?
7. Zu welchem Paket und Modul gehört die Klasse?
8. Welche direkten Unterklassen hat die Klasse?
9. Nennen Sie eine Methode der Klasse zur Überprüfung, ob ein String nur aus Whitespace-Zeichen besteht oder leer ist.
10. Was bewirkt die Methode `public String translateEscapes()`?
11. Nennen Sie eine überladene Methode der Klasse, es genügt, nur den Namen der Methode anzugeben.
12. Hat die Klasse öffentliche Klassenmethoden?
13. Hat die Klasse öffentliche Objektmethoden?
14. Hat die Klasse öffentliche Konstruktoren?
15. Hat die Klasse öffentliche Klassenattribute?
16. Hat die Klasse öffentliche Objektattribute?

b) (*, Die Klasse **Math**, 7 Minuten)

1. Wie lautet der vollständige Kopf der Klasse?
2. Beschreiben Sie in einem Satz die Klasse.
3. Ist die Klasse final?
4. Ist die Klasse abstrakt?
5. Welche Schnittstellen implementiert die Klasse (direkt und indirekt)?
6. Welche Oberklasse hat die Klasse?
7. Zu welchem Paket und Modul gehört die Klasse?
8. Welche Unterklassen hat die Klasse?
9. Hat die Klasse öffentliche Klassenmethoden?
10. Hat die Klasse öffentliche Objektmethoden?
11. Hat die Klasse öffentliche Konstruktoren?
12. Hat die Klasse öffentliche Klassenattribute?
13. Hat die Klasse öffentliche Objektattribute?
14. Mit welcher Methode kann man den nächstgrößeren **double**-Wert einer gegebenen Zahl in eine Richtung berechnen?

c) (*, Die Schnittstelle **Iterable**<T>, 3 Minuten)

1. Wie lautet der vollständige Kopf der Schnittstelle?
2. Gibt es Unterschnittstellen der Schnittstelle?
3. Gibt es Oberschnittstellen der Schnittstelle?
4. Nennen Sie eine abstrakte Methode der Schnittstelle (vollen Kopf angeben).
5. Hat die Schnittstelle statische Methoden?
6. Wie viele Default-Methoden hat die Schnittstelle?